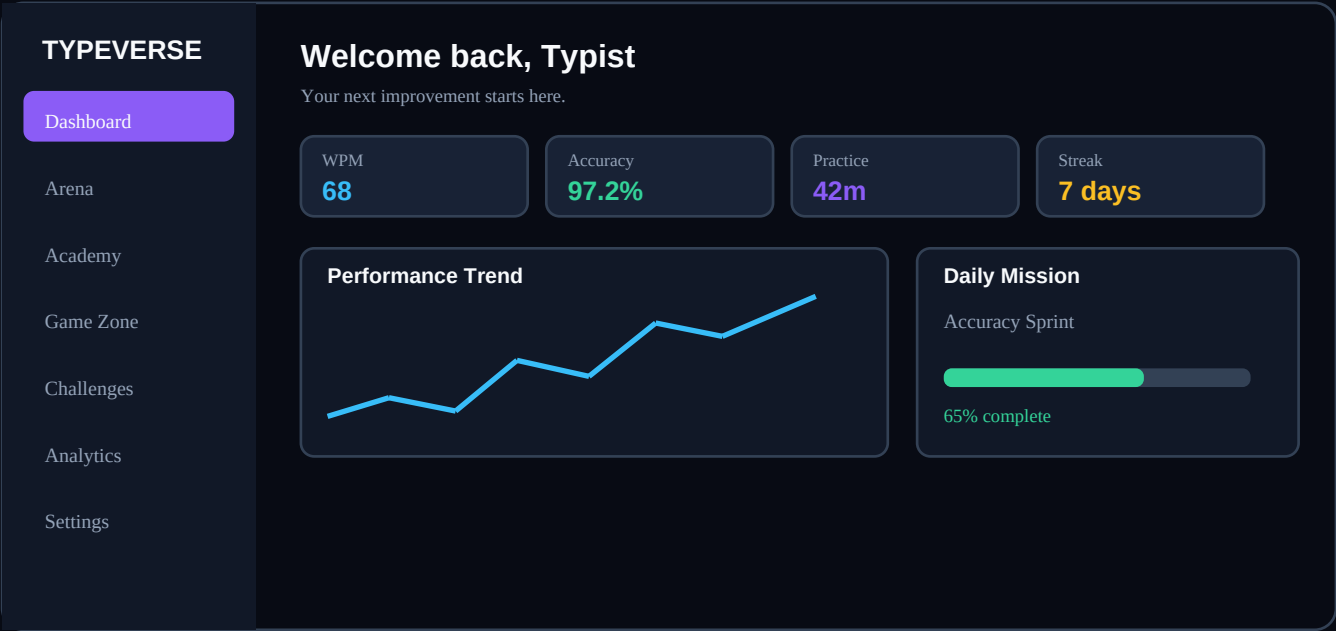


TYPEVERSE

Type. Train. Compete. Evolve.

Premium Hybrid Typing Platform



Complete Product Specification · UI Structure · PRD · Architecture · API · Database
· Roadmap

Executive Overview

TYPEVERSE is a full-stack, desktop-first typing improvement platform combining professional typing tools with immersive game experiences. It is designed to help users improve speed, accuracy, consistency, and confidence through structured learning, adaptive practice, analytics, and game-based challenges.

Core product decision: TYPEVERSE has two connected experiences—Focus Mode for professional typing and Immersive Mode for game-based challenges.

Dimension	Decision
Product style	Hybrid: professional typing + immersive game experience
Build target	Desktop-first; minimum initial design target 1280 × 720
Backend	Full-stack with secure authentication and PostgreSQL
Visual direction	Premium glassmorphism with restrained gaming accents
Primary users	Students, beginner typists, competitive typists, developers and professionals

Product Goals

- Provide accurate typing tests and meaningful metrics.
- Teach touch typing through structured lessons.
- Recommend targeted practice using measured performance.
- Make repetition engaging with missions, achievements, and games.
- Support secure user accounts and persistent progress.
- Deliver a polished, accessible, low-latency desktop experience.

Non-goals for the first release

- Full real-time multiplayer infrastructure.
- Complete conversational AI tutor.
- Every keyboard layout and language.
- Large-scale classroom administration.
- Monetization before core user value is validated.

Market & Product Positioning

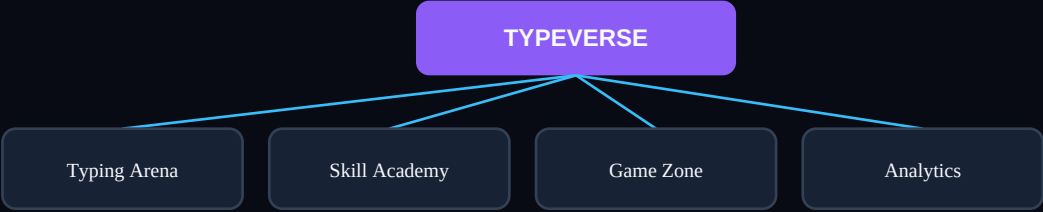
The major typing platforms tend to specialize in speed testing, touch-typing instruction, adaptive practice, competition, or gamification. TYPEVERSE combines these patterns into one coherent product while keeping the typing experience focused.

Platform / Category	Patterns worth studying	TYPEVERSE opportunity
Monkeytype	Minimalist tests, customization, statistics	Keep the focused test experience; add missions and adaptive improvement.
Keybr	Generated practice around weak letters	Explain weak-key recommendations and make drills rewarding.
TypingClub / Typing.com	Structured lessons and education	Add a flexible personal learning journey.
TypeRacer	Competitive typing races	Add challenges and races after core scoring is reliable.
Nitro Type	Game-based racing and rewards	Create a broader game world with multiple modes.
10FastFingers	Fast tests, languages, competition	Connect tests to a structured improvement loop.
Ratatype / TypeBlast	Lessons, certificates, story/game patterns	Use progression, practice goals, and optional certificates later.

Market Insights

- Different users need different practice: beginners need technique; experienced users need consistency, speed, or difficult text.
- Feedback should lead to improvement, not only display a WPM number.
- Competition and challenges can encourage repeat usage.
- Personalization should be based on actual performance rather than identical tests for everyone.
- Gamification already exists in parts of the market; TYPEVERSE differentiates through integration and execution.

Information Architecture



Main Navigation

Route	Purpose
Dashboard	Personal overview, missions, recommendations, recent sessions
Typing Arena	Time, words, quotes, code, and custom tests
Skill Academy	Progressive lessons, finger placement, weak-key drills
Game Zone	TypeQuest, Falling Words, Survival Mode
Challenge Hub	Daily challenges, personal challenges, friends and races
Analytics	WPM trends, accuracy, weak-key and session analysis
Profile & Settings	Account, preferences, themes, sound, accessibility

Application Route Tree

TYPEVERSE → Landing Page → Authenticated App → Dashboard / Arena / Academy / Game Zone / Challenges / Analytics / Settings

Navigation Requirements

- Persistent desktop sidebar with active route indicator.
- Keyboard-accessible navigation and visible focus states.
- Collapsible sidebar for smaller desktop widths.
- Clear separation between focused typing screens and game screens.
- Profile and notification access in the top navigation.

Premium UI Design System

The interface should feel like a high-quality productivity application that transitions naturally into an immersive game environment.

Token	Suggested value	Usage
Background	#080B14	Main application background
Surface	#111827	Panels and cards
Elevated surface	#182235	Modals and elevated elements
Primary accent	#8B5CF6	Primary actions
Secondary accent	#38BDF8	Progress and highlights
Success	#34D399	Correct typing and completed goals
Error	#FB7185	Typing errors and validation
Primary text	#F8FAFC	Main content
Muted text	#94A3B8	Secondary content

Typography & Components

- UI font: Inter or Geist Sans.
- Typing font: readable monospace font with configuration.
- Use tabular numerals for WPM, accuracy, and statistics.
- Use glass surfaces selectively; avoid excessive blur.
- Buttons need default, hover, focus, loading, and disabled states.
- Use restrained animations that communicate state changes.
- Provide reduced-motion support and accessible contrast.

Design Rule

The interface should look impressive in screenshots while remaining comfortable during a 15–30 minute practice session. The active typing surface must remain stable and distraction-free.

UI Structure & Page Specifications

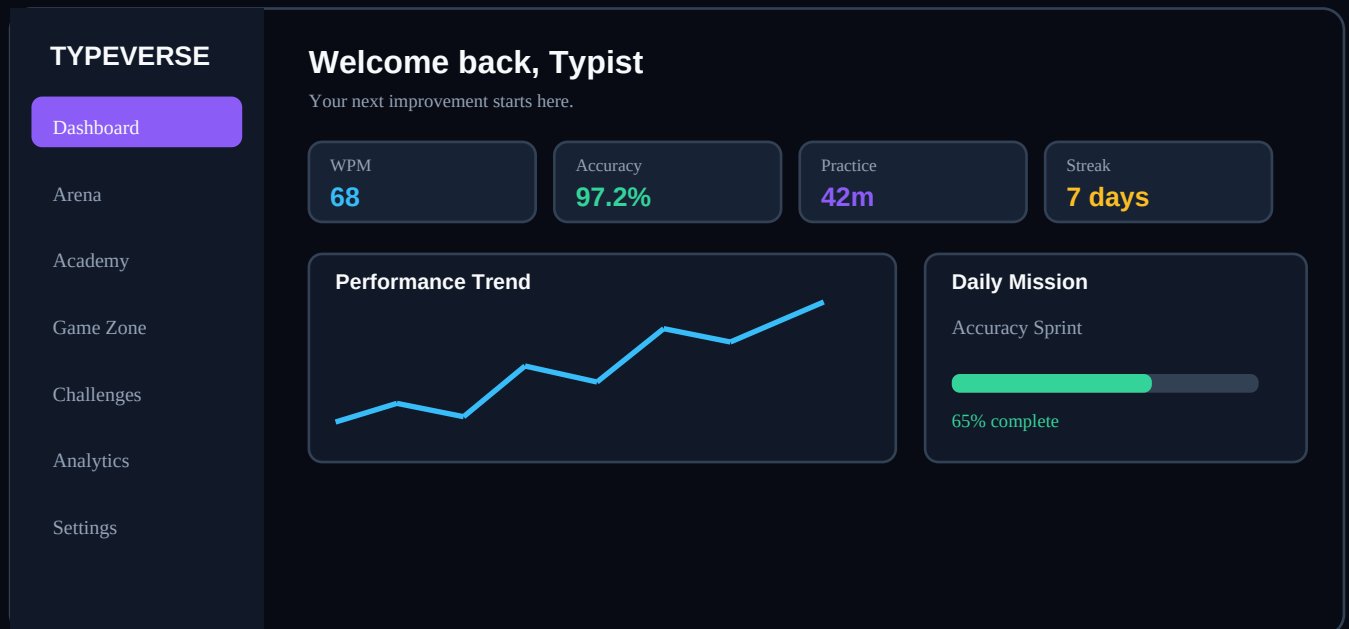
Landing Page

Purpose: explain the product and get users into a test quickly.

Header: TYPEVERSE | Features | Learn | Challenges | Sign In | Start

- Premium hero section with animated but lightweight background.
- Interactive keyboard preview or typing demonstration.
- Primary CTA: Start Typing.
- Secondary CTA: Explore Games.
- Feature highlights and trust-building content.
- No fabricated live statistics.

Dashboard



- Show WPM, accuracy, practice time, and streak.
- Continue Learning card and Daily Mission card.
- Performance trend chart and recent sessions.
- Quick-start test action.
- Loading skeletons, empty states, and API error handling.

Typing Arena

TYPING ARENA

TIME · 60s

WPM

68

Accuracy

97.2%

Progress

42%

The future belongs to people who practice with purpose.

The future belongs to people who practice with purpose.

A

S

D

F

J

K

L

;

Interaction flow



- Time modes: 15s, 30s, 60s, and 120s.
- Word-count mode, quotes/passages, code mode, and custom tests.
- Highlight current character and mark errors clearly.
- Use a dedicated input handler and safe keyboard event handling.
- Define backspace, composition, unsupported keys, and timer policy.
- Prevent duplicate submission and clean up timers on unmount.

Results Page

Result component	Requirement
WPM	Display standard metric and define calculation policy
Accuracy	Show percentage and explain whether it is gross or adjusted
Duration	Show exact test duration
Errors	Show count and key/pattern breakdown when sufficient data exists
Actions	Retry, targeted practice, dashboard
Persistence	Submit only after validation; recover cleanly from API failure

Skill Academy

Level 1: Home Row → Level 2: Top Row → Level 3: Bottom Row → Level 4: Speed & Consistency

- Home-row introduction and finger-placement guidance.
- Letter-specific drills and accuracy-focused exercises.
- Locked, available, in-progress, and completed lesson states.
- Completion thresholds should be configurable and tested.
- Users can repeat previously completed lessons.
- Each lesson must state its learning objective.

Game Zone

Game	Gameplay concept	Technical requirements
Falling Words	Words appear; correct typing earns points; misses affect game state	Reuse typing input/scoring; deterministic generation for tests; separate rendering and rules
TypeQuest	Mission-based world with precision, speed, endurance, weak-key, and mastery objectives	Configurable missions; XP; achievement rewards; verified results
Survival Mode	Endless or timed typing challenge with escalating difficulty	State machine; difficulty progression; replay and game-over states

Challenge Hub

- Daily challenge with defined objective and completion state.
- Personal-best challenge replay.
- Friend challenges and multiplayer races as later features.
- Leaderboards only after server-side score validation.

Functional Requirements

FR-01: Typing Test Engine

- Start, pause where applicable, restart, and finish.
- Time and word-count tests.
- Passage rendering with current-character feedback.
- Keyboard input handling.
- WPM, accuracy, and error tracking.
- Result summary and reliable session completion.

WPM Convention

WPM = (corrected characters ÷ 5) ÷ time in minutes. The product must document whether raw WPM, adjusted WPM, and incorrect characters are treated differently.

FR-02: Typing Lessons

- Beginner lesson path.
- Home-row and finger placement.
- Progressive exercises.
- Accuracy gates and completion states.

FR-03: Adaptive Practice

- Track errors by key and patterns.
- Use sufficient data before labeling a weakness.
- Generate targeted exercises.
- Explain why a recommendation was made.
- Avoid over-targeting a key after one isolated error.

FR-04: Dashboard

- Best and current WPM.
- Recent accuracy and practice summary.
- Daily mission and quick start.
- Recent results and recommendation.

FR-05: Analytics

- WPM and accuracy history.
- Session count and duration.
- Weak-key summary.
- Best performance records.
- Clear metric definitions.

Functional Requirements — Continued

FR-06: Gamification

- XP or progress points.
- Daily missions.
- Achievement badges.
- Level progression.
- Cosmetic rewards.
- Personal challenge replays.

FR-07: Game Zone

- Falling Words.
- Survival mode.
- Difficulty progression.
- Score tracking.
- Game-over and replay flow.
- Keyboard input integration.

FR-08: Customization

- Light and dark themes.
- At least three visual themes.
- Sound effects toggle.
- Reduced motion preference.
- Responsive supported desktop layouts.
- Accessible contrast.

FR-09: Accessibility

- Keyboard navigation.
- Visible focus states.
- Screen-reader-friendly labels.
- Reduced motion support.
- Do not rely on color alone.
- Physical keyboard preferred for speed comparisons.

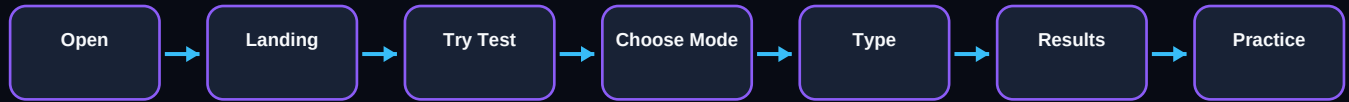
Non-functional Requirements

Category	Requirement
Performance	Responsive input handling and minimal typing latency
Reliability	Stable timer, scoring, restart behavior
Accessibility	Keyboard navigation, focus, contrast, reduced motion
Security	Server-side validation; client scores not trusted
Privacy	Collect only necessary data

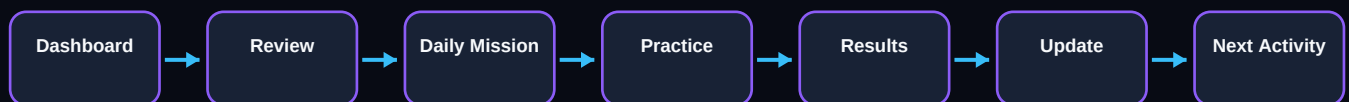
Category	Requirement
Scalability	Separate typing engine from accounts and social features
Maintainability	Modular components and documented schemas

User Flows & Diagrams

First-Time User Flow



Returning User Flow



Adaptive Learning Flow



Adaptive algorithm

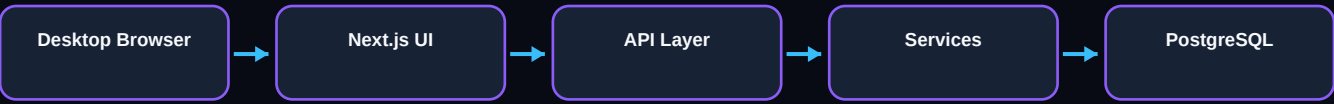
$W_k = aE_k + bT_k + cR_k$, where E_k is error rate, T_k is a time/difficulty signal, R_k is recency-weighted performance, and $a/b/c$ are tunable weights. This is a proposed starting model that must be validated.

Recommendation rules

- If a key pattern has low accuracy and sufficient attempts, recommend targeted practice.
- If accuracy improves and errors decline, reduce its priority.
- If data is insufficient, recommend general practice.
- Allow users to dismiss or skip recommendations.

Full-Stack Architecture

Architecture layers



Recommended stack

Layer	Technology
Frontend	Next.js, React, TypeScript
Styling	Tailwind CSS
Backend	Next.js Route Handlers or modular API layer
Database	PostgreSQL
ORM	Prisma or Drizzle
Authentication	Secure session-based authentication
Validation	Zod or equivalent
Charts	Recharts or lightweight custom charts
Deployment	Vercel + managed PostgreSQL
Testing	Unit, integration, and browser-based tests

Backend decision

Use Next.js with a modular backend layer for the first full-stack release. A separate FastAPI service can be added later for advanced analytics, machine learning, or independently scaled APIs.

Suggested project structure

```
typeverse/\n  app/\n    page.tsx\n  arena/\n  academy/\n  games/\n  analytics/\n  components/\n  lib/\n  typing-engine/\n  scoring/\n  adaptive-practice/\n  storage/\n  data/\n  types/\n  public/
```

Database Design

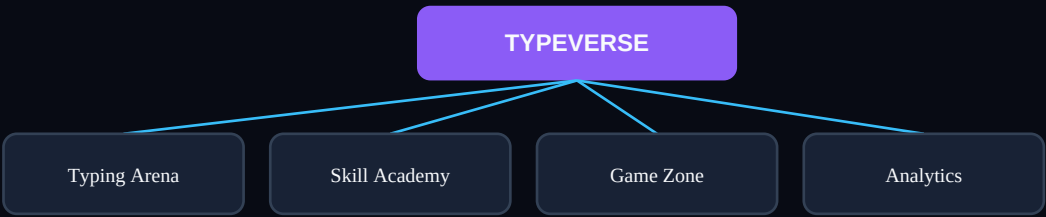
Core entities

Entity	Key fields
User	id, username, email, authProviderId/passwordHash, displayName, avatarUrl, timestamps
TypingSession	id, userId, mode, duration, passageld, WPM, accuracy, errors, character counts, timestamps, status
KeyMetric	id, sessionId, key, attempts, errors, average latency
LessonProgress	id, userId, lessonId, status, best accuracy, attempts, completedAt
MissionProgress	id, userId, missionId, progress value, status, completedAt
UserSetting	id, userId, theme, sound, reduced motion, typing preferences
Achievement	id, name, criteria, reward
UserAchievement	userId, achievementId, awardedAt

Database rules

- Use foreign keys and indexes.
- Store timestamps consistently.
- Validate metric ranges.
- Define whether completed sessions can be edited.
- Separate authentication data from public profile data.
- Never store plaintext passwords.
- Avoid storing unnecessary raw keystroke content.

Entity relationship summary



API Specification

Authentication endpoints

Method	Endpoint	Purpose
POST	/api/auth/register	Create account
POST	/api/auth/login	Authenticate
POST	/api/auth/logout	End session
GET	/api/auth/me	Retrieve current user

Typing session endpoints

Method	Endpoint	Purpose
GET	/api/typing/config	Available modes and settings
POST	/api/sessions	Submit completed session
GET	/api/sessions	List user sessions
GET	/api/sessions/:id	View session details
POST	/api/sessions/:id/retry	Optional retry workflow

Practice and game endpoints

Method	Endpoint	Purpose
GET	/api/practice/recommendations	Retrieve recommendations
POST	/api/practice/drills	Create/start drill
GET	/api/practice/weak-keys	Weak-key summary
POST	/api/practice/drills/:id/complete	Record drill completion
GET	/api/games	List games
POST	/api/games/sessions	Create game session
POST	/api/games/sessions/:id/complete	Submit game result
GET	/api/challenges/daily	Get daily challenge

Security requirement

Validate every payload server-side. Use authorization checks for user-owned resources. Do not trust client-submitted scores for leaderboards. Apply rate limits to sensitive endpoints.

Authentication, Security & Privacy

User roles

Role	Access
Guest	Try tests, view temporary results, explore landing page
Registered User	Save progress, analytics, lessons, achievements, challenges
Admin	Manage passages, lessons, missions, platform metrics, reports

Security checklist

- Secure password hashing or trusted provider.
- Secure cookies or carefully designed token strategy.
- Session expiration and logout.
- Server-side input validation.
- Authorization on every private resource.
- Rate limiting and injection protection.
- Secrets in environment variables.
- Privacy-conscious logging.

Leaderboard integrity

- Store session metadata and define scoring policy.
- Validate modes and durations.
- Detect obviously invalid submissions.
- Use server-side eligibility checks.
- Provide a review/removal process for suspicious results.
- Communicate how scores are validated.

Privacy principles

- Collect only data required for the feature.
- Do not store unnecessary raw keystrokes.
- Offer account deletion and data removal workflows.
- Separate private account information from public profile fields.

Analytics & Success Metrics

Metric	Purpose
Test completion rate	Measures whether users finish tests
Practice sessions per user	Indicates repeat usage
7-day retention	Measures return behavior
Lesson completion	Tracks learning activity
Daily mission completion	Measures engagement
Game replay rate	Measures interest in game modes
Average accuracy	Tracks typing outcome
WPM trend	Tracks speed over time

Learning-related metrics

- Accuracy.
- WPM under comparable test conditions.
- Consistency.
- Error rate on targeted keys.
- Performance on unseen passages.
- Improvement without accuracy decline.

Validation plan

Compare performance across consistent test modes and time periods. Do not claim that the product improves typing ability until meaningful user data supports that conclusion.

Risks & Mitigation

Risk	Mitigation
Gamification overshadows learning	Give accuracy, consistency, and targeted practice meaningful roles
Incorrect scoring	Unit-test timer, WPM, accuracy, backspace, and edge cases
Scope becomes too large	Ship core testing and lessons before multiplayer
Poor input experience	Optimize event handling and test with real keyboards
Weak recommendations	Use sufficient data, confidence thresholds, and transparent explanations

Gamification & Reward System

XP and levels

Level	Example requirement
Level 1	Complete onboarding
Level 2	Complete 3 practice sessions
Level 3	Complete 5 lessons
Level 4	Complete daily challenges
Level 5	Earn mastery achievements

Reward types

- XP and badges.
- Cosmetic themes.
- Avatar frames and profile titles.
- Mission unlocks.
- Game-world progression.

Achievement examples

Achievement	Condition
First Steps	Complete first typing session
Precision Master	Meet an accuracy target under defined conditions
Consistent Practice	Practice on a defined number of separate days

Reward design principle

Rewards should encourage practice without incentivizing users to sacrifice accuracy for speed. Cosmetic rewards should not distort typing scores or learning outcomes.

Roadmap & Release Strategy

Phase	Deliverables
1 · Foundation	Project setup, design system, authentication, database, typing engine, session persistence, dashboard, basic lessons, tests
2 · Learning	Adaptive practice, analytics, daily missions, achievements, additional modes, recommendation feedback
3 · Game Experience	Falling Words, TypeQuest, Survival Mode, game progression, cosmetics
4 · Social	Friend challenges, leaderboards, multiplayer races, server-validated scoring, privacy settings

MVP priority matrix

Must Have	Should Have	Could Have
Typing engine	Daily missions	Multiplayer races
WPM and accuracy	Story mode	Cloud leaderboards
Beginner lessons	Falling Words	Advanced social profiles
Local / cloud session records	Achievements	Complex competitive economy
Adaptive drills	Personal-best replay	Expanded language support

Recommended release strategy

Release a functional typing and learning experience first. Use real user feedback to decide which expansion features deserve priority. Avoid waiting for every game and social feature before validating the core product.

MVP Acceptance Checklist

- User can register and log in securely.
- User can complete a time-based typing test.
- WPM and accuracy calculations are tested.
- Typing sessions can be saved and retrieved.
- User can view their dashboard.
- User can complete beginner lessons.
- User can view basic performance analytics.
- Weak-key practice works with sufficient data.
- API validation and authorization are implemented.
- Core interface supports keyboard navigation.
- Loading and error states are handled.
- Deployment and database configuration are documented.

QA requirements

- Unit tests for scoring, timer, backspace, and input edge cases.
- Integration tests for session submission and retrieval.
- Browser tests for login, test completion, results, and navigation.
- Accessibility checks for focus, contrast, labels, and reduced motion.
- Performance checks for typing latency and unnecessary re-renders.

Definition of Done

A feature is done only when its UI, state behavior, backend validation, persistence, accessibility, error handling, and tests are implemented and reviewed.

Master AntiGravity Build Prompt

Use the following condensed implementation directive as the primary build brief. It preserves the complete product direction while giving AntiGravity a clear development order.

```
ROLE: Senior full-stack engineer + product designer + UX architect + game systems designer + QA engineer.\n\nPRODUCT: TYPEVERSE – premium desktop-first hybrid typing platform.\n\nSTYLE: Premium glassmorphism with restrained gaming accents.\n\nBACKEND: Full-stack, PostgreSQL, secure authentication, server validation.\n\nBUILD ORDER:\n1. Project setup and design system\n2. Authentication and database\n3. Typing engine\n4. Typing Arena\n5. Session persistence\n6. Dashboard\n7. Skill Academy\n8. Analytics\n9. Adaptive Practice\n10. Daily Missions\n11. Game Zone\n12. Security review\n13. Testing\n14. Deployment preparation\n\nQUALITY:\nFunctional, premium-looking, desktop-optimized, secure, accessible, maintainable, testable.\n\nRULES:\nDo not use fake user statistics.\nDo not claim placeholder features are complete.\nDo not add unnecessary AI APIs.\nDo not compromise typing performance for visual effects.\nKeep the typing engine independent from UI and games.
```

Final Product Principle

Build a reliable core product first, then expand the immersive experience. The best TYPEVERSE experience is one where users can clearly see how their practice improves their skill—and where the game layer makes them want to return.

TYPEVERSE

Type. Train. Compete. Evolve.